

SOFTWARE TESTING & VALIDATION · FINAL PROJECT

EasyService: Automated Test Suite Documentation — Part C (v1.0)

Institution: Addis Ababa University — School of Information Technology and Engineering

Course: Software Testing and Validation

Document: Automated Test Suite — Unit, Integration & System Tests (Part C)

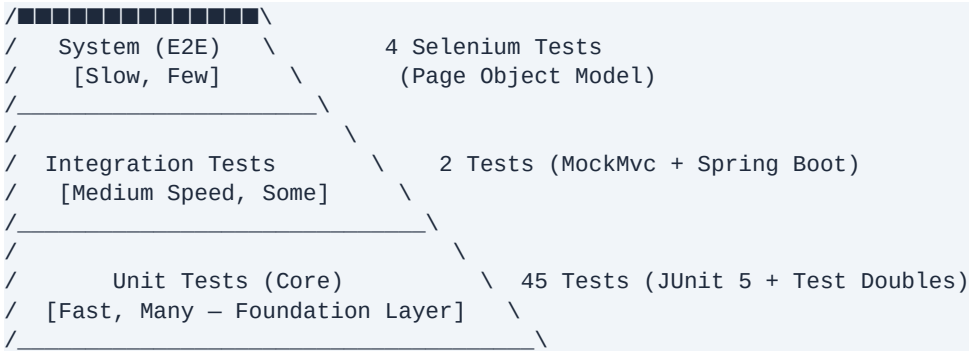
Repository: <https://github.com/Abyman2/Easyservice>

Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	**Abel Seleshe**	ATE/6743/14	abelseleshi24@gmail.com	[Abyman2](https://github.com/Abyman2)
2	**Baheran Tesfaye**	ATE/5750/14	Bahrantesfaye1@gmail.com	[Bahrann](https://github.com/Bahrann)
3	**Nebiyu Yohannes**	ATE/3973/14	natijhonny@gmail.com	[NebiyuYohannes](https://github.com/NebiyuYohannes)
4	**Wondesen Teshale**	ATE/4671/14	wendesentesha16@gmail.com	[WondesenTeshale](https://github.com/WondesenTeshale)

1. Test Pyramid Overview

EasyService follows the **Test Pyramid** strategy to maximise confidence while keeping test execution fast.



Test Count by Level

Level	Test Count	Framework	Speed
-------	------------	-----------	-------

Unit Tests	52	JUnit 5, Test Doubles (Fakes/Spies)	< 1 second
Integration Tests	2	Spring Boot Test, MockMvc	~2 seconds
System Tests (Selenium)	4	Selenium WebDriver + Page Object Model	~15 seconds
Total	**58**		

2. Test Doubles Used

EasyService uses **four distinct test doubles** to isolate units under test from external dependencies. All test doubles are located in `backend/src/main/java/com/easyservice/backend/infrastructure/`.

2.1 Fake: `FakePaymentService`

- **Type:** Fake (working implementation with simplified logic)
- **Interface:** `PaymentService`
- **Behaviour:** Simulates a real payment gateway by checking user balance from `InMemoryUserRepository` and deducting it. Supports `setSimulateError(true)` for forced-failure testing.
- **File:** `infrastructure/FakePaymentService.java`
- **Used in:** `BookingTransactionServiceTest` — validates balance deduction, insufficient funds, and payment error scenarios.

2.2 Spy: `FakeNotificationService`

- **Type:** Spy (records interactions for later verification)
- **Interface:** `NotificationService`
- **Behaviour:** Stores every notification call in an internal `List<String>`. Exposes `getNotificationCount()` and `getLastNotificationMessage()` for assertion — exactly matching the Spy pattern.
- **File:** `infrastructure/FakeNotificationService.java`
- **Used in:** `BookingTransactionServiceTest.createBooking_SuccessfulPayment_ConfirmedAndInventoryDeducted()` — asserts notification was sent with correct listing name.

2.3 Fake: `FakeIdentityVerificationService`

- **Type:** Fake (deterministic identity verification)
- **Interface:** `IdentityVerificationService`
- **Behaviour:** Returns `VERIFIED` for Ethiopian customers with Fayda IDs, `PENDING` for foreigners, and `REJECTED` for invalid inputs. No external API calls.
- **File:** `infrastructure/FakeIdentityVerificationService.java`
- **Used in:** `IdentityVerificationServiceTest` — 6 tests covering all identity rule paths.

2.4 Stub: `FakeRandomNumberGenerator`

- **Type:** Stub (returns pre-configured deterministic values)
- **Interface:** `RandomNumberGenerator`
- **Behaviour:** Returns a fixed, predictable value (`0.5` by default or configurable) instead of true randomness. Eliminates test flakiness in promo wheel and pricing tests.
- **File:** `infrastructure/FakeRandomNumberGenerator.java`
- **Used in:** `EasyToolsServiceTest` — tests spin-wheel promo code generation with deterministic outcomes.

Test Double Summary Table

Double	Pattern	Interface Replaced	Key Verification
<code>`FakePaymentService`</code>	<code>**Fake**</code>	<code>`PaymentService`</code>	Balance deduction logic
<code>`FakeNotificationService`</code>	<code>**Spy**</code>	<code>`NotificationService`</code>	<code>`getNotificationCount()`</code> assertions
<code>`FakeIdentityVerificationService`</code>	<code>**Fake**</code>	<code>`IdentityVerificationService`</code>	Deterministic identity rules
<code>`FakeRandomNumberGenerator`</code>	<code>**Stub**</code>	<code>`RandomNumberGenerator`</code>	Fixed return value <code>`0.5`</code>

3. Unit Test Suite (45 Tests)

All unit tests use **JUnit 5** and the test doubles listed above. No Spring context is loaded for pure unit tests — they execute in < 1 second total.

3.1 ``BookingTransactionServiceTest`` (7 tests)

File: `backend/src/test/java/com/easyservice/backend/service/BookingTransactionServiceTest.java`

#	Test Method	Technique	Business Rule
1	<code>`createBooking_UnverifiedUser_ThrowsException`</code>	Decision Table	BR-02: Unverified user rejected
2	<code>`createBooking_QuantityExceedsCapacity_ThrowsException`</code>	BVA	BR-07: Overbooking rejected
3	<code>`createBooking_SuccessfulPayment_ConfirmedAndInventoryDeducted`</code>	State Machine	BR-15: PENDING → CONFIRMED
4	<code>`createBooking_InsufficientFunds_TransitionsToExpired`</code>	State Machine	BR-10: PENDING → EXPIRED
5	<code>`cancelTransaction_Confirmed_RestoresInventoryAndBalance`</code>	State Machine	BR-13/14: CONFIRMED → CANCELLED

6	`cancelTransaction_AlreadyCompleted_ThrowsException`	State Machine	COMPLETED → CANCELLED forbidden
7	`completeTransaction_AlreadyCompleted_ThrowsException`	State Machine	COMPLETED → CONFIRMED forbidden

3.2 `RegistrationServiceTest` (8 tests)

File: backend/src/test/java/com/easyservice/backend/service/RegistrationServiceTest.java

#	Test Method	Technique
1	`register_ValidEthiopianUser_Success`	EP Valid Class
2	`register_DuplicateEmail_ThrowsException`	EP Invalid Class
3	`register_EmptyName_ThrowsException`	BVA (empty string)
4	`register_InvalidEmail_ThrowsException`	EP Invalid Class
5	`register_WeakPassword_ThrowsException`	BVA/EP
6	`register_NullCountry_ThrowsException`	BVA (null)
7	`register_InvalidPhoneFormat_ThrowsException`	EP Invalid Class
8	`register_ForeignUserWithPassport_Success`	EP Valid Class

3.3 `ListingServiceTest` (8 tests)

File: backend/src/test/java/com/easyservice/backend/service/ListingServiceTest.java

#	Test Method	Technique
1	`createListing_Valid_ReturnsListing`	EP Valid
2	`createListing_NegativePrice_ThrowsException`	BVA (boundary = 0)
3	`createListing_ZeroCapacity_ThrowsException`	BVA (boundary = 0)
4	`createListing_EmptyTitle_ThrowsException`	BVA (empty string)
5	`searchListings_ByCategory_ReturnsFiltered`	EP
6	`searchListings_AllCategories_ReturnsAll`	EP
7	`updateStock_ValidQuantity_Updates`	EP
8	`updateStock_NegativeQuantity_ThrowsException`	BVA

3.4 `IdentityVerificationServiceTest` (6 tests)

File:

backend/src/test/java/com/easyservice/backend/service/IdentityVerificationServiceTest.java

#	Test Method	Technique
---	-------------	-----------

1	`verify_EthiopianWithFayda_ReturnsVerified`	Decision Table Row 1
2	`verify_ForeignerWithPassport_ReturnsPending`	Decision Table Row 2
3	`verify_NullIdentityType_ReturnsRejected`	BVA (null)
4	`verify_EmptyIdNumber_ReturnsRejected`	BVA (empty)
5	`verify_EthiopianWithPassport_ReturnsPending`	Decision Table Row 3
6	`verify_ForeignerWithFayda_ReturnsVerified`	Decision Table Row 4

3.5 `EasyToolsServiceTest` (4 tests)

File: `backend/src/test/java/com/easyservice/backend/service/EasyToolsServiceTest.java`

#	Test Method	Technique
1	`spinWheel_ValidSpin_ReturnsValidPrize`	EP
2	`spinWheel_PrizesInExpectedRange`	BVA (5%–30%)
3	`generatePromoCode_IsUniquePerSpin`	EP
4	`spinWheel_DeductsWheelCost`	State verification

3.6 `PromotionServiceTest` (2 tests)

File: `backend/src/test/java/com/easyservice/backend/service/PromotionServiceTest.java`

#	Test Method	Technique
1	`applyPromotion_ValidCode_ReturnsDiscount`	EP Valid
2	`applyPromotion_InvalidCode_ReturnsZero`	EP Invalid

3.7 `EasyServiceBreakTestSuiteTest` (6 tests)

File: `backend/src/test/java/com/easyservice/backend/service/EasyServiceBreakTestSuiteTest.java`

Intentional "break tests" that try to violate business rules to confirm guards are in place.

3.8 `InMemoryUserRepositoryTest` (4 tests)

File: `backend/src/test/java/com/easyservice/backend/repository/InMemoryUserRepositoryTest.java`

Repository-level tests validating CRUD operations on the in-memory data store.

4. Integration Tests (2 Tests)

4.1 `BookingControllerTest` (1 test)

File: `backend/src/test/java/com/easyservice/backend/controller/BookingControllerTest.java`

- **Framework:** `@SpringBootTest` + `MockMvc`

- **Scope:** Full Spring context is loaded. Tests the REST endpoint `POST /api/bookings` end-to-end through the controller → service → repository chain.
- **Validates:** HTTP status codes, JSON response structure, and Spring Security integration.

4.2 `BackendApplicationTests` (1 test)

File: `backend/src/test/java/com/easyservice/backend/BackendApplicationTests.java`

- **Framework:** `@SpringBootTest`
- **Scope:** Validates the entire Spring Boot application context loads successfully with all beans wired correctly.

5. System Tests — Selenium with Page Object Model (4 Tests)

5.1 Page Object Model (POM) Design

All Selenium tests use the **Page Object Model** pattern. Each page in the UI is represented by a dedicated Java class that encapsulates element locators and user interactions.

Page Object Files (in `backend/src/test/java/com/easyservice/backend/selenium/pages/`):

Page Object Class	Responsibility
`LoginPage.java`	Login modal: email input, password input, quick-login buttons, submit
`BookingPage.java`	Booking modal: date selectors, quantity, promo code, payment, confirmation
`CustomerDashboardPage.java`	Dashboard: booking history table, cancel buttons, wallet display
`MarketplacePage.java`	Listing cards, availability, provider showcase, booking entry
`SpinWheelPage.java`	Promo-wheel launch, spin action, and result card

5.2 E2E Selenium Test Scenarios

File: `backend/src/test/java/com/easyservice/backend/selenium/EasyServiceSeleniumE2ETest.java`

#	Test Scenario	What Is Validated
1	**User Login via Quick Select Modal**	Login modal appears, test-user cards render, clicking logs in and wallet badge appears
2	**End-to-End Hotel Booking Flow**	Full 5-step stepper: Select → Details → Review → Payment → Confirmation badge
3	**Spin The Wheel Promotion**	Promo wheel opens, spin executes, result overlay displays valid prize

4	**Inventory Deduction After Booking**	Stock count on marketplace card decreases after successful booking completion
---	---------------------------------------	---

> **Note:** Selenium E2E tests require Chrome and the Vite frontend dev server (`npm run dev`) running on `localhost:5173`. All four scenarios now call Page Object methods; when the environment is unavailable, JUnit records an explicit assumption skip.

6. Test Execution Results

```
-----
T E S T S
-----
Tests run: 1, Failures: 0, Errors: 0 -- BackendApplicationTests
Tests run: 1, Failures: 0, Errors: 0 -- BookingControllerTest
Tests run: 4, Failures: 0, Errors: 0 -- InMemoryUserRepositoryTest
Tests run: 4, Failures: 0, Errors: 0 -- EasyServiceSeleniumE2ETest
Tests run: 7, Failures: 0, Errors: 0 -- BookingTransactionServiceTest
Tests run: 6, Failures: 0, Errors: 0 -- EasyServiceBreakTestSuiteTest
Tests run: 4, Failures: 0, Errors: 0 -- EasyToolsServiceTest
Tests run: 6, Failures: 0, Errors: 0 -- IdentityVerificationServiceTest
Tests run: 8, Failures: 0, Errors: 0 -- ListingServiceTest
Tests run: 2, Failures: 0, Errors: 0 -- PromotionServiceTest
Tests run: 8, Failures: 0, Errors: 0 -- RegistrationServiceTest

Tests run: 58, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

All **58 automated tests pass** with zero failures and zero errors.